```
"""Basic elements with one input and/or output"""

from __future__ import division

import random, math

import utils, simtime
from connect import Element, Pump, Inject


class Car(simtime.Timed):
    """Represents a car; knows total existance and waiting time"""
    def __init__(self):
        super(Car, self).__init__()
        self.total = self.waited = 0
        if hasattr(self, 'city'):
            if not hasattr(self.city, 'car_id'):
                self.city.car_id = 0
            self.id = self.city.car_id
            self.city.car_id += 1
        else:
            self.id = -1
    def tick(self):
        self.total += 1
    def wait(self):
        """Called by a queue when the car stays in it for this cycle"""
        self.waited += 1
        return self
    def __str__(self):
        return ("<Car %s: total = %2s, waited = %2s>" %
                (self.id, self.total, self.waited))


class Source(simtime.Timed, Inject):
    """Abstract class for car sources."""
    def __init__(self):
        super(Source, self).__init__()
        self.inputs = {}                # a one-sided element


class ProbSource(Source):
    """Every cycle, create a car with given probability."""
    def __init__(self, prob):
        super(ProbSource, self).__init__()
        self.prob = prob
    def tick(self):
        if random.random() < self.prob:
            self.target.append(self.city.Car())


# A server is completely active (pumps and injects as he decides).
class Server(simtime.Timed, Inject, Pump):
    """Abstract class for single input / single output servers."""
    pass


# For demonstraion only - simplest server test...
class ProbServer(Server):
    """Serve each cycle with given probability."""
    def __init__(self, prob):
        super(ProbServer, self).__init__()
        self.prob = prob
    def tick(self):
        if random.random() < self.prob:
            if len(self.queue) != 0:
                self.target.append(self.queue.pop(0))


# The interface is modeled after Python lists.  Makes most sense for
# the queue, "half a list" for other cases.
```

```
# A queue is completely passive (tick() is only for accounting).
# Thus a queue can be a merge/split point for traffic.
class Queue(simtime.Timed, Element):
    """List-like place for cars that tells them they are waiting."""
    def __init__(self):
        super(Queue, self).__init__()
        self.q = []
    # Simple delegation.
    def __getattr__(self, name):
        return getattr(self.q, name)
    # builtins like `len()` doesn't understand `__getattr__`...
    def __len__(self):
        return len(self.q)
    def __getitem__(self, *args):
        return self.q.__getitem__(*args)
    def tick(self):
        for car in self.q:
            car.wait()


# Collect the stats...
class StatQueue(Queue):
    """A queue that also collects waiting statistics"""
    def __init__(self):
        super(StatQueue, self).__init__()
        self.waited = utils.Statistics()
    def append(self, car):
        self.q.append(car)
        car.waited_here = 0
    def pop(self, *args):
        car = self.q.pop(*args)
        self.waited.submit(car.waited_here)
        return car
    def tick(self):
        super(StatQueue, self).tick()
        for car in self.q:
            car.waited_here += 1
    def __str__(self):
        return ("<%s: waited here: %s>" %
                (self.__class__.__name__, self.waited))


class LoadStatQueue(StatQueue):
    """Abstract class for a StatQueue that also measures incoming load."""
    def __init__(self):
        super(LoadStatQueue, self).__init__()
        self.load = 0
    def append(self, car):
        super(LoadStatQueue, self).append(car)
        self.load += 1
    def tick(self):
        super(LoadStatQueue, self).tick()
        self.load = 0
    def estimate_load(self):
        """Return estimate of average load, between 0 and 1."""
        raise NotImplementedError


class LoadHistQueue(LoadStatQueue):
    """A queue that also collects load history and waiting statistics.

    Load is estimated by averaging over a rectangular window.

    """
    def __init__(self, hist_len = None):
        """`hist_len`, if given, constarains load history length."""
        super(LoadHistQueue, self).__init__()
        self.hist_len = hist_len
        self.load_hist = []
```

```
    def tick(self):
        super(LoadHistQueue, self).tick()
        self.load_hist.append(self.load)
        if self.hist_len:
            del self.load_hist[:-self.hist_len]
        self.load = 0
    def estimate_load(self):
        """Return estimate of average load, between 0 and 1."""
        from operator import add
        return (len(self.load_hist) and
                reduce(add, self.load_hist) / len(self.load_hist))


class LoadExpQueue(LoadStatQueue):
    """A queue that also collects load history and waiting statistics.

    Load is estimated by exponential average.

    """
    def __init__(self, exp_len = 10):
        """`exp_len` is the characteristic decay length of the average."""
        super(LoadExpQueue, self).__init__()
        self.decay = math.exp(-1/exp_len)
        self.load_average = 0
    def tick(self):
        super(LoadExpQueue, self).tick()
        self.load_average = (self.load_average * self.decay +
                             self.load * (1 - self.decay))
        self.load = 0
    def estimate_load(self):
        """Return estimate of average load, between 0 and 1."""
        return self.load_average


class Drain(Queue):
    """Eats all entering car.

    Subclasses can implement self.process(car) to do extra things with
    each car.

    """
    def __init__(self):
        super(Drain, self).__init__()
        self.outputs = {}               # a one-sided element
    def process(self, car):
        pass
    def tick(self):
        for car in self.q:
            self.process(car)
            car.retire()
            del self.q[:]  # eat them :-\


class StatDrain(Drain):
    """Eat all entering cars, collecting statistics."""
    def __init__(self):
        super(StatDrain, self).__init__()
        self.waited_to_total = utils.Statistics()
    def process(self, car):
        if car.total:
            self.waited_to_total.submit(car.waited / car.total)
    def __str__(self):
        return ("<%s: waited/total: %s>" %
                (self.__class__.__name__, self.waited_to_total))


class PrintDrain(Drain):
    """Eat cars, also printing them.  Too verbose for long runs."""
    def process(self, car):
        print car
```

```
class Road(simtime.Timed, Inject):
    """Constant delay.  If 0, works instantly."""
    def __init__(self, delay):
        super(Road, self).__init__()
        self.left = {}
        self.delay = delay
    def append(self, car):
        if self.delay == 0:
            self.target.append(car)
        else:
            self.left[car] = self.delay
    def tick(self):
        for car in self.left.keys():
            self.left[car] -= 1
            if self.left[car] == 0:
                self.target.append(car)
                del self.left[car]
```